
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 3

Desenvolvimento Estruturado de Programas em C

Exercícios (3.11–3.46)

Resolvidos passo a passo, com código completo
Abordagem incremental Deitel

Contents

1	Exercício 3.11 – Identificar e Corrigir Erros	2
2	Exercício 3.12 – Preenchimento de Lacunas	3
3	Exercício 3.13 – O que o programa imprime?	3
4	Exercício 3.14 – Instruções em Pseudocódigo	4
5	Exercício 3.15 – Algoritmos em Pseudocódigo	5
6	Exercício 3.16 – Verdadeiro ou Falso	6
7	Exercício 3.17 – Consumo de Gasolina	6
8	Exercício 3.18 – Limite de Crédito	8
9	Exercício 3.19 – Comissão de Vendas	9
10	Exercício 3.20 – Calculadora de Juros Simples	10
11	Exercício 3.21 – Calculadora de Salário com Hora Extra	11
12	Exercício 3.22 – Pré vs. Pós-Decremento	12
13	Exercício 3.23 – Imprimindo 1 a 10 com Loop	13
14	Exercício 3.24 – Achar o Maior de 10 Números	14
15	Exercício 3.25 – Tabela N, 10N, 100N, 1000N	15
16	Exercício 3.26 – Tabela A, A+2, A+4, A+6	15
17	Exercício 3.27 – Os Dois Maiores de 10	17
18	Exercício 3.28 – Validação de Entrada	18
19	Exercício 3.29 – Saída do Programa (Operador Condicional)	19
20	Exercício 3.30 – Saída do Programa (Loops Aninhados)	19
21	Exercício 3.31 – Problema do Else Pendurado	20
22	Exercício 3.32 – Outro Problema do Else Pendurado	22
23	Exercício 3.33 – Quadrado de Asteriscos	23
24	Exercício 3.34 – Quadrado Vazio	23
25	Exercício 3.35 – Testador de Palíndromo	25
26	Exercício 3.36 – Binário para Decimal	26

27 Exercício 3.37 – Velocidade do Computador	27
28 Exercício 3.38 – 100 Asteriscos com Quebra a cada 10	27
29 Exercício 3.39 – Contando Dígitos 7	28
30 Exercício 3.40 – Padrão de Tabuleiro de Xadrez	29
31 Exercício 3.41 – Múltiplos de 2 com Loop Infinito	29
32 Exercício 3.42 – Círculo com float	30
33 Exercício 3.43 – O Que Está Errado	31
34 Exercício 3.44 – Lados de um Triângulo	31
35 Exercício 3.45 – Triângulo Retângulo	32
36 Exercício 3.46 – Fatorial, Constante e e e^x	33

1. Exercício 3.11 – Identificar e Corrigir Erros

Resposta detalhada

a) Código com erro:

```
1  if ( idade >= 65 );
2      printf( "Idade e maior ou igual a 65\n" );
3  else
4      printf( "Idade e menor que 65\n" );
```

Erro: Ponto e vírgula após `)` do `if` cria um corpo vazio, e o `else` fica sem um `if` associado, gerando erro de sintaxe.

Correção:

```
1  if ( idade >= 65 ) {
2      printf( "Idade e maior ou igual a 65\n" );
3  }
4  else {
5      printf( "Idade e menor que 65\n" );
6  }
```

b) Código com erro:

```
1  int x = 1, total;
2  while ( x <= 10 ) {
3      total += x;
4      ++x;
5  }
```

Erro: A variável `total` não foi inicializada – contém “lixo”. A soma incluirá esse valor desconhecido.

Correção:

```
1  int x = 1, total = 0;  /* inicializa total em 0 */
2  while ( x <= 10 ) {
3      total += x;
4      ++x;
5  }
```

c) Código com erro:

```
1  while ( x <= 100 )
2      total += x;
3      ++x;
```

Erros: (1) Sem chaves, apenas a primeira instrução faz parte do laço; `++x` fica fora do `while`, criando loop infinito.

Correção:

```
1  while ( x <= 100 ) {
2      total += x;
3      ++x;
4  }
```

d) Código com erro:

```
1 while ( y > 0 ) {  
2     printf( "%d\n", y );  
3     ++y;  
4 }
```

Erro: ++y aumenta y – a condição $y > 0$ nunca se tornará falsa para y positivo. Loop infinito.

Correção: Trocar ++y por -y.

```
1 while ( y > 0 ) {  
2     printf( "%d\n", y );  
3     --y;      /* decrementa em vez de incrementar */  
4 }
```

2. Exercício 3.12 – Preenchimento de Lacunas

Resposta detalhada

- a) Uma série de ações em uma **ordem** específica.
- b) Sinônimo de procedimento: **algoritmo**.
- c) Variável que acumula soma de números: **total** (ou acumulador).
- d) Definir variáveis com valores específicos no início: **inicialização**.
- e) Valor especial para indicar fim de entrada: **sentinela**, **sinalizador**, **artificial**, **flag**.
- f) Representação gráfica de um algoritmo: **fluxograma**.
- g) Em fluxograma, ordem indicada por **linhas de fluxo** (setas).
- h) Símbolo de finalização indica **início** e **fim** de cada algoritmo.
- i) Cálculos por instruções de atribuição; entrada/saída por **scanf** e **printf**.
- j) Item escrito dentro de símbolo de decisão: **condição**.

3. Exercício 3.13 – O que o programa imprime?

Enunciado 3.13

Determine a saída do programa que calcula $y = x * x$ para x de 1 a 10 e acumula **total**.

Resposta detalhada

O programa calcula os quadrados de 1 a 10 e exibe cada um em uma linha. Ao final, imprime a soma de todos os quadrados.

Cálculo da soma: $1^2 + 2^2 + \dots + 10^2 = 1 + 4 + 9 + 16 + 25 + 36 + 49 + 64 + 81 + 100 = 385$.

Saída do programa

```
1
2
4
9
16
25
36
49
64
81
100
Total is 385
```

Nota: A mensagem final tem “Total is” (em inglês) tal como aparece no código original do exercício.

4. Exercício 3.14 – Instruções em Pseudocódigo

Resposta detalhada

a) Apresentar mensagem 'Digite dois números':

Imprimir “Digite dois números”

b) Atribuir soma de x, y, z a p:

Definir p como a soma de x, y e z

c) Testar se $m > 2v$ em estrutura if...else:

*Se m for maior que duas vezes o valor atual de v
...ações se verdadeiro
senão
...ações se falso*

d) Obter valores para s, r, t pelo teclado:

Ler s, r e t do teclado

5. Exercício 3.15 – Algoritmos em Pseudocódigo

Resposta detalhada

a) Soma de dois números:

```
Imprimir "Digite dois números:"  
Ler n1 e n2 do teclado  
Definir soma como  $n1 + n2$   
Imprimir soma
```

b) Maior de dois números:

```
Imprimir "Digite dois números:"  
Ler n1 e n2 do teclado  
Se  $n1 > n2$   
    Imprimir "O maior é" n1  
senão se  $n2 > n1$   
    Imprimir "O maior é" n2  
senão  
    Imprimir "Os números são iguais"
```

c) Soma de série positiva com sentinela -1:

```
Definir total como zero  
Imprimir "Digite o primeiro número (-1 para terminar):"  
Ler número do teclado  
Enquanto número for diferente de -1  
    Somar número ao total  
    Imprimir "Digite o próximo número (-1 para terminar):"  
    Ler próximo número  
Imprimir "A soma é" total
```

6. Exercício 3.16 – Verdadeiro ou Falso

Resposta detalhada

- a) **Falso.** A parte mais difícil é *compreender o problema e desenvolver o algoritmo*. A escrita do código em C é geralmente a parte mais simples, uma vez que o algoritmo está bem definido.
- b) **Verdadeiro.** O valor de sentinela deve ser escolhido de modo a nunca poder ser confundido com um dado válido.
- c) **Falso.** Linhas de fluxo *não* indicam ações; elas indicam a *ordem* em que as ações (representadas por retângulos) são executadas. Ações são indicadas pelos símbolos retangulares.
- d) **Falso.** Condições em símbolos de decisão contêm operadores *relacionais* (<, >, <=, >=) e *de igualdade* (==, !=), *não* aritméticos. Também podem ser qualquer expressão que avalie para zero (falso) ou diferente de zero (verdadeiro).
- e) **Verdadeiro.** Cada nível de refinamento top-down é uma representação completa do algoritmo – apenas o nível de detalhe varia.

7. Exercício 3.17 – Consumo de Gasolina

Resposta detalhada

Algoritmo (pseudocódigo):

```
Inicializar km_total e litros_total como zero
Solicitar litros do primeiro abastecimento
Enquanto litros não for -1
    Solicitar km dirigidos
    Calcular km/litro do abastecimento atual
    Imprimir consumo atual
    Acumular km_total e litros_total
    Solicitar litros do próximo abastecimento
Se litros_total > 0
    Imprimir consumo geral = km_total / litros_total
```

```
1  /* Ex3.17: consumo_gasolina.c
2     Calcula consumo de combustivel por abastecimento e
3     total */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      float litros;          /* litros de cada
10         abastecimento */
11      float km;             /* km dirigidos por
```



```

    abastecimento */
9   float kmTotal = 0.0; /* km totais dirigidos
    */
10  float litrosTotal = 0.0; /* litros totais
    consumidos */
11  float consumoAtual; /* km/L do abastecimento
    atual */
12  float consumoGeral; /* km/L do total
    */

13
14  printf( "Informe quantos litros abasteceu (-1 para
    terminar): " );
15  scanf( "%f", &litros );
16
17  while ( litros != -1.0 ) {
18      printf( "Informe quantos km dirigiu: " );
19      scanf( "%f", &km );
20
21      consumoAtual = km / litros;
22      printf( "O consumo atual e de %f km/l\n",
        consumoAtual );
23
24      kmTotal += km;
25      litrosTotal += litros;
26
27      printf( "Informe quantos litros abasteceu (-1
        para terminar): " );
28      scanf( "%f", &litros );
29  } /* fim do while */
30
31  if ( litrosTotal > 0.0 ) {
32      consumoGeral = kmTotal / litrosTotal;
33      printf( "O consumo geral foi de %f km/l\n",
        consumoGeral );
34  }
35  else {
36      printf( "Nenhum abastecimento foi informado.\n
        " );
37  }
38
39  return 0;
40 } /* fim da funcao main */
```

Observação: Note o uso de float (Cap. 3) para permitir valores como 25.6 litros. Também, evitamos divisão por zero testando `litrosTotal > 0` antes de calcular o consumo geral.

8. Exercício 3.18 – Limite de Crédito

Resposta detalhada

```
1  /* Ex3.18: limite_credito.c
2     Verifica se cliente excedeu o limite de credito */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int conta;                /* numero da conta
8                                */
9     float saldoInicial;       /* saldo no inicio do mes
10                                */
11     float encargos;           /* total de encargos
12                                */
13     float creditos;           /* total de creditos
14                                */
15     float limite;             /* limite de credito
16                                */
17     float novoSaldo;          /* novo saldo apos calculos
18                                */
19
20     printf( "Informe o numero da conta (-1 para
21             terminar): " );
22     scanf( "%d", &conta );
23
24     while ( conta != -1 ) {
25         printf( "Informe o saldo inicial: " );
26         scanf( "%f", &saldoInicial );
27
28         printf( "Informe o total de encargos: " );
29         scanf( "%f", &encargos );
30
31         printf( "Informe o total de creditos: " );
32         scanf( "%f", &creditos );
33
34         printf( "Informe o limite de credito: " );
35         scanf( "%f", &limite );
36
37         novoSaldo = saldoInicial + encargos - creditos
38             ;
39
40         if ( novoSaldo > limite ) {
41             printf( "Conta: %d\n", conta );
42             printf( "Limite de credito: %.2f\n",
43                     limite );
44             printf( "Saldo: %.2f\n", novoSaldo );
45             printf( "Limite de credito ultrapassado.\n
46                     " );
47         }
48     }
49 }
```

```
37         } /* fim do if */
38
39         printf( "\nInforme o numero da conta (-1 para
40                 terminar): " );
41         scanf( "%d", &conta );
42     } /* fim do while */
43
44     return 0;
45 } /* fim da funcao main */
```

9. Exercício 3.19 – Comissão de Vendas

Resposta detalhada

```
1  /* Ex3.19: comissao_vendas.c
2     Calcula salario semanal de vendedor com 9% de
3     comissao */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      float vendas;      /* vendas brutas da semana
10                          */
11      float salario;     /* salario total a receber
12                          */
13      float salarioBase = 200.00; /* salario fixo
14                                  semanal */
15      float comissao     = 0.09;  /* taxa de comissao 9%
16                                  */
17
18      printf( "Informe a venda em reais (-1 para
19              terminar): " );
20      scanf( "%f", &vendas );
21
22      while ( vendas != -1.0 ) {
23          salario = salarioBase + ( vendas * comissao );
24          printf( "O pagamento e de: R$ %.2f\n", salario
25                  );
26
27          printf( "\nInforme a venda em reais (-1 para
28                  terminar): " );
29          scanf( "%f", &vendas );
30      } /* fim do while */
31
32      return 0;
33 } /* fim da funcao main */
```

10. Exercício 3.20 – Calculadora de Juros Simples

Resposta detalhada

Fórmula: $\text{juros} = \frac{\text{principal} \times \text{taxa} \times \text{dias}}{365}$

```
1  /* Ex3.20: juros_simples.c
2     Calcula juros simples para varios emprestimos */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     float principal;    /* valor do emprestimo
8                          */
9     float taxa;         /* taxa de juros anual (
10                          decimal) */
11     int dias;           /* prazo do emprestimo em dias
12                          */
13     float juros;        /* juros calculados
14                          */
15
16     printf( "Informe o valor principal do emprestimo "
17            "(-1 para terminar): " );
18     scanf( "%f", &principal );
19
20     while ( principal != -1.0 ) {
21         printf( "Informe a taxa de juros: " );
22         scanf( "%f", &taxa );
23
24         printf( "Informe o prazo do emprestimo em dias
25                : " );
26         scanf( "%d", &dias );
27
28         juros = principal * taxa * dias / 365.0;
29         printf( "O valor dos juros e de R$ %.2f\n",
30                juros );
31
32         printf( "\nInforme o valor principal do
33                emprestimo "
34                "(-1 para terminar): " );
35         scanf( "%f", &principal );
36     } /* fim do while */
37
38     return 0;
39 } /* fim da funcao main */
```

11. Exercício 3.21 – Calculadora de Salário com Hora Extra

Resposta detalhada

Regra de pagamento:

- Até 40 horas: $\text{salário} = \text{horas} \times \text{valor_hora}$
- Acima de 40 horas: $\text{salário} = (40 \times \text{valor_hora}) + (\text{extras} \times 1,5 \times \text{valor_hora})$

```
1  /* Ex3.21: salario.c
2     Calcula salario semanal com hora extra (1,5x acima
3     de 40h) */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      float horas;           /* horas trabalhadas na
10                             semana */
11      float valorHora;       /* valor do salario por hora
12                             */
13      float salario;         /* salario calculado
14                             */
15
16      printf( "Digite # de horas trabalhadas (-1 para
17              terminar): " );
18      scanf( "%f", &horas );
19
20      while ( horas != -1.0 ) {
21          printf( "Digite o salario por hora do
22                  funcionario "
23                  "(R$00,00): " );
24          scanf( "%f", &valorHora );
25
26          if ( horas <= 40.0 ) {
27              salario = horas * valorHora;
28          } /* fim do if */
29          else {
30              /* Horas normais (40) + horas extras (1,5x
31              ) */
32              salario = ( 40.0 * valorHora ) +
33                      ( ( horas - 40.0 ) * valorHora *
34                        1.5 );
35          } /* fim do else */
36
37          printf( "Salario e de R$ %.2f\n", salario );
38
39          printf( "\nDigite # de horas trabalhadas "
40                  "(-1 para terminar): " );
41          scanf( "%f", &horas );
42      } /* fim do while */
43
44 }
```

```
35     return 0;
36 } /* fim da funcao main */
```

12. Exercício 3.22 – Pré vs. Pós-Decremento

Resposta detalhada

```
1  /* Ex3.22: pre_vs_pos_decremento.c
2     Demonstra a diferenca entre --x (pre) e x-- (pos)
3     */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int c;
10
11     /* === Demonstracao do POS-decremento === */
12     c = 5;
13     printf( "=== Pos-decremento ===\n" );
14     printf( "Valor inicial de c: %d\n", c );
15     printf( "Imprime c-- (usa o valor 5, depois
16             decrementa): %d\n", c-- );
17     printf( "Valor de c agora: %d\n\n", c );
18
19     /* === Demonstracao do PRE-decremento === */
20     c = 5;
21     printf( "=== Pre-decremento ===\n" );
22     printf( "Valor inicial de c: %d\n", c );
23     printf( "Imprime --c (decrementa, depois usa o
24             valor 4): %d\n", --c );
25     printf( "Valor de c agora: %d\n", c );
26
27     return 0;
28 } /* fim da funcao main */
```

Saída do programa

```
=== Pos-decremento ===
Valor inicial de c: 5
Imprime c- (usa o valor 5, depois decrementa): 5
Valor de c agora: 4

=== Pre-decremento ===
Valor inicial de c: 5
Imprime --c (decrementa, depois usa o valor 4): 4
Valor de c agora: 4
```

13. Exercício 3.23 – Imprimindo 1 a 10 com Loop

Resposta detalhada

```
1  /* Ex3.23: numeros_1_a_10_lado_a_lado.c
2     Imprime 1 a 10 lado a lado, separados por 3 espacos
3     */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int i = 1;
10
11     while ( i <= 10 ) {
12         printf( "%d  ", i ); /* numero seguido de 3
13                                espacos */
14         ++i;
15     } /* fim do while */
16
17     printf( "\n" ); /* nova linha ao final */
18
19     return 0;
20 } /* fim da funcao main */
```

Saída do programa

```
1  2  3  4  5  6  7  8  9  10
```

14. Exercício 3.24 – Achar o Maior de 10 Números

Resposta detalhada

Pseudocódigo:

```
Inicializar contador como 1
Ler o primeiro número e armazenar em maior
Enquanto contador for menor que 10
    Ler próximo número
    Se número > maior
        Atribuir número a maior
    Incrementar contador
Imprimir o valor de maior
```

```
1  /* Ex3.24: maior_de_10.c
2     Encontra o maior valor entre 10 numeros lidos */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int contador = 1;  /* conta numeros lidos (1 a 10)
8                          */
9      int numero;        /* numero da entrada atual
10                          */
11     int maior;          /* maior valor encontrado
12                          */
13
14     printf( "Digite o 1o numero: " );
15     scanf( "%d", &maior );  /* primeiro e o maior
16                             inicial */
17
18     while ( contador < 10 ) {
19         ++contador;
20         printf( "Digite o %do numero: ", contador );
21         scanf( "%d", &numero );
22
23         if ( numero > maior ) {
24             maior = numero;
25         } /* fim do if */
26     } /* fim do while */
27
28     printf( "\nO maior numero e: %d\n", maior );
29
30     return 0;
31 } /* fim da funcao main */
```


15. Exercício 3.25 – Tabela N, 10N, 100N, 1000N

Resposta detalhada

```
1  /* Ex3.25: tabela_potencias_10.c
2     Imprime tabela com N, 10*N, 100*N, 1000*N para N de
3     1 a 10 */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int n = 1;
10
11     printf( "N\t10*N\t100*N\t1000*N\n" ); /*
12         cabecalho com tabulacoes */
13
14     while ( n <= 10 ) {
15         printf( "%d\t%d\t%d\t%d\n", n, 10 * n, 100 * n
16             , 1000 * n );
17         ++n;
18     } /* fim do while */
19
20     return 0;
21 } /* fim da funcao main */
```

Saída do programa

```
N\t10*N\t100*N\t1000*N
1      10      100      1000
2      20      200      2000
...
10     100     1000     10000
```

16. Exercício 3.26 – Tabela A, A+2, A+4, A+6

Resposta detalhada

```
1  /* Ex3.26: tabela_progressao.c
2     Imprime tabela com A, A+2, A+4, A+6 para A = 3, 6,
3     9, 12, 15 */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int a = 3;
10
11     printf( "A\tA+2\tA+4\tA+6\n" );
```

```
11     while ( a <= 15 ) {  
12         printf( "%d\t%d\t%d\t%d\n", a, a + 2, a + 4, a  
13             + 6 );  
14         a += 3;      /* incrementa A em 3 */  
15     } /* fim do while */  
16  
17     return 0;  
18 } /* fim da funcao main */
```

17. Exercício 3.27 – Os Dois Maiores de 10

Resposta detalhada

Estratégia: Manter duas variáveis – maior (1º lugar) e segundoMaior (2º lugar). Para cada novo número:

- Se for maior que maior: o antigo maior vira segundoMaior, e o novo número vira maior.
- Senão, se for maior que segundoMaior: substitui segundoMaior.

```
1  /* Ex3.27: dois_maiores.c
2     Encontra os dois maiores entre 10 numeros */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int contador = 1;
8      int numero;
9      int maior;
10     int segundoMaior;
11
12     printf( "Digite o 1o numero: " );
13     scanf( "%d", &maior );
14
15     printf( "Digite o 2o numero: " );
16     scanf( "%d", &numero );
17
18     /* Inicializa os dois maiores comparando os dois
19        primeiros */
19     if ( numero > maior ) {
20         segundoMaior = maior;
21         maior = numero;
22     }
23     else {
24         segundoMaior = numero;
25     }
26
27     contador = 2;
28     while ( contador < 10 ) {
29         ++contador;
30         printf( "Digite o %do numero: ", contador );
31         scanf( "%d", &numero );
32
33         if ( numero > maior ) {
34             segundoMaior = maior;
35             maior = numero;
36         }
37         else if ( numero > segundoMaior ) {
38             segundoMaior = numero;
39         }
40     } /* fim do while */
```

```
41
42     printf( "\n0 maior e: %d\n", maior );
43     printf( "0 segundo maior e: %d\n", segundoMaior );
44
45     return 0;
46 } /* fim da funcao main */
```

18. Exercício 3.28 – Validação de Entrada

Resposta detalhada

Conceito: “Looping defensivo” – continuar pedindo a entrada até que o usuário forneça um valor válido (1 ou 2).

```
1  /* Ex3.28: valida_entrada.c
2     Valida que o usuario digite apenas 1 ou 2 */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int resultado;
8
9     printf( "Digite o resultado (1=aprovado, 2=
10             reprovado): " );
11     scanf( "%d", &resultado );
12
13     /* Continua pedindo enquanto a entrada for
14        invalida */
15     while ( resultado != 1 && resultado != 2 ) {
16         printf( "Entrada invalida! Digite 1 ou 2: " );
17         scanf( "%d", &resultado );
18     } /* fim do while */
19
20     if ( resultado == 1 ) {
21         printf( "Aluno aprovado.\n" );
22     }
23     else {
24         printf( "Aluno reprovado.\n" );
25     }
26
27     return 0;
28 } /* fim da funcao main */
```

Nota: O operador && (“e” lógico) será estudado formalmente no Capítulo 4. Ele combina duas condições – ambas precisam ser verdadeiras.

19. Exercício 3.29 – Saída do Programa (Operador Condicional)

Resposta detalhada

O programa usa o operador condicional `?:` para alternar entre duas strings: `contador % 2 ? "****" : "++++++"`.

- Quando contador é **ímpar**, `contador % 2` é 1 (verdadeiro) $\Rightarrow$ imprime `****`
- Quando contador é **par**, `contador % 2` é 0 (falso) $\Rightarrow$ imprime `++++++`

Saída do programa

```
****
++++++
****
++++++
****
++++++
****
++++++
****
++++++
```

20. Exercício 3.30 – Saída do Programa (Loops Aninhados)

Resposta detalhada

O programa tem dois `while` aninhados. O externo controla `linha` (de 10 a 1, decrementando); o interno controla `coluna` (de 1 a 10).

Para cada linha:

- Se `linha` é ímpar: imprime `< 10` vezes
- Se `linha` é par: imprime `> 10` vezes

Como `linha` começa em 10 (par) e decrementa:

Saída do programa

```
>>>>>>>>
<<<<<<<<<
>>>>>>>>
<<<<<<<<<
>>>>>>>>
<<<<<<<<<
>>>>>>>>
<<<<<<<<<
>>>>>>>>
<<<<<<<<<
```

21. Exercício 3.31 – Problema do Else Pendurado

Enunciado 3.31

Determine a saída do código para (x=9, y=11) e (x=11, y=9). *Regra:* o compilador associa o **else** ao **if** **anterior**, a menos que chaves indiquem o contrário.

Resposta detalhada

Item (a): Sem chaves, o **else** é associado ao **if** interno (if (y > 10)), independente do recuo. Reescrevendo com a estrutura real:

```
1  if ( x < 10 ) {  
2      if ( y > 10 ) {  
3          printf( "*****\n" );  
4      } else {  
5          printf( "#####\n" );  
6      }  
7  }  
8  printf( "$$$$$\n" ); /* SEMPRE executa! Esta fora dos  
   ifs */
```

Caso x=9, y=11: x<10 verdadeiro → entra no if externo. y>10 verdadeiro → imprime *****. Depois, sempre imprime \$\$\$\$\$.

Saída do programa

```
*****  
$$$$$
```

Caso x=11, y=9: x<10 falso → pula todo o if externo. Imprime apenas \$\$\$\$\$.

Saída do programa

```
$$$$$
```

Item (b): As chaves mudam a estrutura. Agora o **else** pertence ao **if externo** (if (x < 10)):

```
1  if ( x < 10 ) {  
2      if ( y > 10 ) {  
3          printf( "*****\n" );  
4      }  
5  }  
6  else {  
7      printf( "#####\n" );  
8      printf( "$$$$$\n" );  
9  }
```

Caso x=9, y=11: x<10 verdadeiro, y>10 verdadeiro → *****.

Saída do programa

```
*****
```

Caso $x=11$, $y=9$: $x<10$ falso $\rightarrow$ entra no `else`.

Saída do programa

```
#####  
$$$$$
```

Boa prática de programação

Sempre use chaves `{ }` ao redor dos corpos de `if` e `else`, mesmo quando contêm uma única instrução. Isso elimina ambiguidades, evita o problema do “*dangling else*” e facilita modificações futuras.

22. Exercício 3.32 – Outro Problema do Else Pendurado

Resposta detalhada

Para cada saída desejada, o código deve ser estruturado com chaves apropriadas (o else liga-se ao if anterior por padrão):

Caso (a) – x=5, y=8 deve produzir @@@@ \$\$\$\$ &&&&:

```

1  if ( y == 8 ) {
2      if ( x == 5 ) {
3          printf( "@@@@\n" );
4      } else {
5          printf( "####\n" );
6      }
7      printf( "$$$$\\n" );
8      printf( "&&&&\\n" );
9  }
```

Caso (b) – x=5, y=8 deve produzir apenas @@@@:

```

1  if ( y == 8 ) {
2      if ( x == 5 ) {
3          printf( "@@@@\n" );
4      } else {
5          printf( "####\n" );
6          printf( "$$$$\\n" );
7          printf( "&&&&\\n" );
8      }
9  }
```

Caso (c) – x=5, y=8 deve produzir @@@@ &&&&:

```

1  if ( y == 8 ) {
2      if ( x == 5 ) {
3          printf( "@@@@\n" );
4      } else {
5          printf( "####\n" );
6          printf( "$$$$\\n" );
7      }
8      printf( "&&&&\\n" );
9  }
```

Caso (d) – x=5, y=7 deve produzir ##### \$\$\$\$ &&&&:

```

1  if ( y == 8 ) {
2      if ( x == 5 ) {
3          printf( "@@@@\n" );
4      }
5  } else {
6      printf( "#####\\n" );
7      printf( "$$$$\\n" );
8      printf( "&&&&\\n" );
9  }
```


23. Exercício 3.33 – Quadrado de Asteriscos

Resposta detalhada

```
1  /* Ex3.33: quadrado_asteriscos.c
2     Imprime quadrado solido de asteriscos com lado n */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int lado;          /* tamanho do lado */
8      int linha;         /* contador de linhas */
9      int coluna;        /* contador de colunas */
10
11     printf( "Digite o lado do quadrado (1 a 20): " );
12     scanf( "%d", &lado );
13
14     linha = 1;
15     while ( linha <= lado ) {          /* loop externo:
16         linhas */
17         coluna = 1;
18         while ( coluna <= lado ) {     /* loop interno:
19             colunas */
20             printf( "*" );
21             ++coluna;
22         }
23         printf( "\n" );                /* nova linha ao
24             final */
25         ++linha;
26     }
27
28     return 0;
29 } /* fim da funcao main */
```

24. Exercício 3.34 – Quadrado Vazio

Resposta detalhada

```
1  /* Ex3.34: quadrado_vazio.c
2     Imprime quadrado oco de asteriscos com lado n */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int lado, linha, coluna;
8
9      printf( "Digite o lado do quadrado (1 a 20): " );
10     scanf( "%d", &lado );
```

```
11
12     linha = 1;
13     while ( linha <= lado ) {
14         coluna = 1;
15         while ( coluna <= lado ) {
16             /* Imprime * apenas nas bordas; espaco no
17              interior */
18             if ( linha == 1 || linha == lado ||
19                 coluna == 1 || coluna == lado ) {
20                 printf( "*" );
21             }
22             else {
23                 printf( " " );
24             }
25             ++coluna;
26         }
27         printf( "\n" );
28         ++linha;
29     }
30     return 0;
31 } /* fim da funcao main */
```

Nota: O operador `||` (“ou” lógico) será estudado no Cap. 4. Aqui, ele testa se a posição atual está em alguma das 4 bordas.

25. Exercício 3.35 – Testador de Palíndromo

Resposta detalhada

Estratégia: Extrair os 5 dígitos de um inteiro de 5 dígitos usando divisão e módulo. Comparar 1º com 5º, e 2º com 4º. O 3º (do meio) não importa.

```
1  /* Ex3.35: palindromo.c
2      Verifica se um numero de 5 digitos e palindromo */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int numero;
8      int d1, d2, d3, d4, d5; /* os 5 digitos
9                               individuais */
10
11     printf( "Digite um inteiro de 5 digitos: " );
12     scanf( "%d", &numero );
13
14     /* Extracao dos digitos (mesma tecnica do Ex 2.30)
15        */
16     d1 = numero / 10000;          /* dezena de milhar
17        */
18     d2 = ( numero / 1000 ) % 10; /* unidade de milhar
19        */
20     d3 = ( numero / 100 ) % 10;  /* centena
21        */
22     d4 = ( numero / 10 ) % 10;   /* dezena
23        */
24     d5 = numero % 10;           /* unidade
25        */
26
27     /* Palindromo: 1o == 5o E 2o == 4o */
28     if ( d1 == d5 && d2 == d4 ) {
29         printf( "%d e palindromo\n", numero );
30     }
31     else {
32         printf( "%d NAO e palindromo\n", numero );
33     }
34
35     return 0;
36 } /* fim da funcao main */
```

26. Exercício 3.36 – Binário para Decimal

Resposta detalhada

Estratégia: Extrair cada dígito binário (0 ou 1) da direita para a esquerda (usando % 10 e / 10). Multiplicar cada dígito pelo seu valor posicional (1, 2, 4, 8, 16, ...) e somar.

```

1  /* Ex3.36: binario_para_decimal.c
2     Converte um numero "binario" (composto de 0s e 1s)
3     para decimal */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int binario;           /* numero "binario"
10         digitado */
11
12     int decimal = 0;        /* equivalente decimal
13         */
14
15     int valorPosicao = 1;    /* 1, 2, 4, 8, 16, ...
16         */
17
18     int digito;             /* digito atual (0 ou 1)
19         */
20
21     printf( "Digite um numero binario: " );
22     scanf( "%d", &binario );
23
24     while ( binario > 0 ) {
25         digito = binario % 10;    /* extrai o ultimo
26             digito */
27         decimal += digito * valorPosicao;
28         valorPosicao *= 2;          /* proxima
29             potencia de 2 */
30         binario /= 10;             /* descarta o
31             ultimo digito */
32     }
33
34     printf( "Equivalente decimal: %d\n", decimal );
35
36     return 0;
37 } /* fim da funcao main */

```

Exemplo: 1101

- Iter 1: dígito=1, decimal=1 × 1 = 1; binario=110, valor=2
- Iter 2: dígito=0, decimal=1 + 0 × 2 = 1; binario=11, valor=4
- Iter 3: dígito=1, decimal=1 + 1 × 4 = 5; binario=1, valor=8
- Iter 4: dígito=1, decimal=5 + 1 × 8 = 13; binario=0

Resultado: $1101_2 = 13_{10}$.

27. Exercício 3.37 – Velocidade do Computador

Resposta detalhada

```
1  /* Ex3.37: velocidade_computador.c
2     Conta de 1 a 300 milhoes, exibindo cada multiplo de
3       100 milhoes */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9     int contador = 1;
10
11     while ( contador <= 300000000 ) {
12         if ( contador % 100000000 == 0 ) {
13             printf( "%d\n", contador );
14         }
15         ++contador;
16     }
17
18     printf( "Concluido!\n" );
19     return 0;
20 } /* fim da funcao main */
```

Como medir: Use o cronômetro do celular (ou o comando `time ./a.out` no Linux). Em CPUs modernas, esse loop deve completar em poucos segundos.

Reflexão: Esse é um excelente exercício para sentir o quão rápido o processador realmente é. Bilhões de operações por segundo!

28. Exercício 3.38 – 100 Asteriscos com Quebra a cada 10

Resposta detalhada

```
1  /* Ex3.38: cem_asteriscos.c
2     Imprime 100 asteriscos, com quebra de linha a cada
3       10 */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9     int contador = 1;
10
11     while ( contador <= 100 ) {
12         printf( "*" );
13         if ( contador % 10 == 0 ) { /* a cada 10
14             asteriscos */
15             printf( "\n" );
16         }
17         ++contador;
18     }
```

```
15     }
16
17     return 0;
18 } /* fim da funcao main */
```

A saída será uma matriz de 10×10 asteriscos.

29. Exercício 3.39 – Contando Dígitos 7

Resposta detalhada

```
1  /* Ex3.39: conta_setes.c
2     Conta quantos dígitos do numero sao 7 */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int numero;
8      int contador = 0;
9      int digito;
10
11     printf( "Digite um inteiro positivo: " );
12     scanf( "%d", &numero );
13
14     while ( numero > 0 ) {
15         digito = numero % 10;
16         if ( digito == 7 ) {
17             ++contador;
18         }
19         numero /= 10;
20     }
21
22     printf( "Quantidade de 7s: %d\n", contador );
23     return 0;
24 } /* fim da funcao main */
```

30. Exercício 3.40 – Padrão de Tabuleiro de Xadrez

Resposta detalhada

Restrição: usar apenas `printf("* ")`, `printf(" ")` e `printf("\n")`.

```
1  /* Ex3.40: tabuleiro_xadrez.c
2     Imprime padrao de tabuleiro com asteriscos
3     alternados */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9
10     int linha = 1;
11     int coluna;
12
13     while ( linha <= 8 ) {
14         coluna = 1;
15
16         /* Linhas pares: começar com espaco */
17         if ( linha % 2 == 0 ) {
18             printf( " " );
19         }
20
21         while ( coluna <= 8 ) {
22             printf( "* " );
23             ++coluna;
24         }
25
26         printf( "\n" );
27         ++linha;
28     }
29
30     return 0;
31 } /* fim da funcao main */
```

31. Exercício 3.41 – Múltiplos de 2 com Loop Infinito

Resposta detalhada

```
1  /* Ex3.41: multiplos_2_infinito.c
2     ATENCAO: este programa contem um loop infinito
3     intencional
4     (para fins didaticos). Pressione Ctrl+C para
5     interromper. */
6  #include <stdio.h>
7
8  int main( void )
9  {
```

```
8      int x = 2;
9
10     while ( 1 ) {                /* condicao sempre
11         verdadeira */
12         printf( "%d\n", x );
13         x *= 2;
14     }
15
16     return 0; /* nunca alcancado */
17 } /* fim da funcao main */
```

O que acontece? O programa imprime 2, 4, 8, 16, 32, ... indefinidamente, até que o valor **x transborde** (*overflow*) os limites do tipo **int**. Em sistemas de 32 bits, após 31 dobras (cerca de $2^{30} = 1.073.741.824$), **x** ultrapassa o máximo de **int** ($\approx 2,1$ bilhões), passando a apresentar valores negativos ou erráticos.

Eventualmente, o loop continua “infinitamente”, mas com valores incorretos – até o usuário interromper com Ctrl+C.

32. Exercício 3.42 – Círculo com float

Resposta detalhada

```
1  /* Ex3.42: circulo_float.c
2     Calcula diametro, circunferencia e area com
3     precisao */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      float raio;
10     float diametro, circunferencia, area;
11     float pi = 3.14159f;
12
13     printf( "Digite o raio do circulo: " );
14     scanf( "%f", &raio );
15
16     diametro      = 2.0f * raio;
17     circunferencia = 2.0f * pi * raio;
18     area          = pi * raio * raio;
19
20     printf( "Diametro:          %.3f\n", diametro );
21     printf( "Circunferencia:    %.3f\n", circunferencia );
22     ;
23     printf( "Area:              %.3f\n", area );
24
25     return 0;
26 } /* fim da funcao main */
```


33. Exercício 3.43 – O Que Está Errado

Enunciado 3.43

```
printf( "%d", ++( x + y ) );
```

Resposta detalhada

Erro: O operador de incremento ++ só pode ser aplicado a um **nome de variável simples** – *não* a uma expressão. $(x + y)$ é uma expressão, não uma variável; portanto, $++(x + y)$ é um **erro de sintaxe**.

Provável intenção: O programador queria imprimir o valor de $x + y + 1$.
Reescrita correta:

```
1 printf( "%d", x + y + 1 );
```

Ou, se quiser *também* incrementar x ou y :

```
1 ++x;           /* incrementa x */
2 printf( "%d", x + y );
```

34. Exercício 3.44 – Lados de um Triângulo

Resposta detalhada

Critério (desigualdade triangular): Três lados a , b , c podem formar um triângulo se, e somente se, a soma de quaisquer dois lados é maior que o terceiro: $a + b > c$ e $a + c > b$ e $b + c > a$.

```
1  /* Ex3.44: lados_triangulo.c
2     Verifica se 3 valores podem formar lados de um
3     triângulo */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      float a, b, c;
10
11     printf( "Digite tres valores nao zero: " );
12     scanf( "%f%f%f", &a, &b, &c );
13
14     if ( a + b > c && a + c > b && b + c > a ) {
15         printf( "Sim, podem formar um triângulo.\n" );
16     }
17     else {
18         printf( "Nao formam um triângulo.\n" );
19     }
20
21     return 0;
```

```
20 } /* fim da funcao main */
```

35. Exercício 3.45 – Triângulo Retângulo

Resposta detalhada

Critério (Teorema de Pitágoras): Em um triângulo retângulo, o quadrado da hipotenusa é igual à soma dos quadrados dos catetos. Como não sabemos qual lado é a hipotenusa, testamos as três possibilidades:

```
1  /* Ex3.45: triangulo_retangulo.c
2     Verifica se 3 inteiros formam um triangulo
3     retangulo */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int a, b, c;
10
11     printf( "Digite tres inteiros nao zero: " );
12     scanf( "%d%d%d", &a, &b, &c );
13
14     if ( a*a + b*b == c*c ||
15         a*a + c*c == b*b ||
16         b*b + c*c == a*a ) {
17         printf( "Sim, podem formar um triangulo
18                 retangulo.\n" );
19     }
20     else {
21         printf( "Nao formam triangulo retangulo.\n" );
22     }
23
24     return 0;
25 } /* fim da funcao main */
```

Exemplos clássicos (ternos pitagóricos): (3, 4, 5), (5, 12, 13), (8, 15, 17).

36. Exercício 3.46 – Fatorial, Constante e e^x

Item (a) – Fatorial

Resposta detalhada

Definição: $n! = n \cdot (n - 1) \cdot (n - 2) \cdots 1$, com $0! = 1$.

```

1  /* Ex3.46(a): fatorial.c
2     Calcula n! para n nao negativo */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int n;
8      int fatorial = 1;
9      int i = 1;
10
11     printf( "Digite um inteiro nao negativo: " );
12     scanf( "%d", &n );
13
14     while ( i <= n ) {
15         fatorial *= i;
16         ++i;
17     }
18
19     printf( "%d! = %d\n", n, fatorial );
20     return 0;
21 } /* fim da funcao main */

```

Atenção: int suporta no máximo 12! ($\approx 4,79 \times 10^8$). Para fatoriais maiores, será preciso usar long (Cap. 4) ou double.

Item (b) – Constante e

Resposta detalhada

Fórmula: $e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots$

```

1  /* Ex3.46(b): constante_e.c
2     Calcula a constante matematica e */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      float e = 1.0f;      /* termo inicial */
8      float termo = 1.0f; /* 1/n! atual */
9      int n = 1;
10
11     while ( n <= 20 ) { /* somar 20 termos para boa
12                         precisao */

```

```

12         termo /= n;          /* divide pelo proximo n para
                                gerar 1/n! */
13         e += termo;
14         ++n;
15     }
16
17     printf( "e = %.6f\n", e );
18     return 0;
19 } /* fim da funcao main */

```

Resultado: $e \approx 2,718282$ (valor real: 2,71828182...).

Item (c) – e^x

Resposta detalhada

Fórmula: $e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$

```

1  /* Ex3.46(c): e_x.c
2     Calcula e elevado a x usando serie de Taylor */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      float x;
8      float resultado = 1.0f;
9      float termo = 1.0f;
10     int n = 1;
11
12     printf( "Digite x: " );
13     scanf( "%f", &x );
14
15     while ( n <= 20 ) {
16         termo = termo * x / n;      /* gera x^n / n!
                                     incrementalmente */
17         resultado += termo;
18         ++n;
19     }
20
21     printf( "e^%.2f = %.6f\n", x, resultado );
22     return 0;
23 } /* fim da funcao main */

```

Truque: Em vez de calcular x^n e $n!$ separadamente (custoso), geramos cada termo a partir do anterior: $\frac{x^n}{n!} = \frac{x^{n-1}}{(n-1)!} \cdot \frac{x}{n}$.